Capstone midterm evaluation

Lecture Autosummary: Grounded Multi-Source Lecture Summaries with Adaptive Comprehension Checks

Shamil Arslanov

Capstone type: Thesis (individual)

Advisor: ???

Problem statement

A student who watches a lecture has no reliable way to review it or to check whether they actually understood it. The two artifacts a course leaves behind are both inadequate: slides lose everything the lecturer only *said* (worked examples, caveats, intuition, "this will be on the exam" emphasis), while the full video recording is 2–3.5 hours long and impractical to re-watch before an exam.

This project builds and — more importantly — *measures* a pipeline that turns a lecture recording plus its slide deck into:

1. a **grounded structured summary** (a "conspect") that merges the spoken lecture with the slides, tagging every statement with its source (`slide` / `transcript` / `both`);
2. an **adaptive self-check quiz** whose questions are automatically retired and regenerated based on measured psychometric quality (difficulty, discrimination) of real student responses.

The central research question is evaluative, not just generative: **can we numerically prove that a multi-source summary is richer than the slide deck alone, and that its content is faithful to the sources?** The thesis metric is *transcript-only coverage* — the share of facts the lecturer spoke but the slides do not show that survive into the summary. For a slides-only summary this is 0% by definition.

Three experiments structure the work:

- **E1 — claim-level summary evaluation:** coverage (recall), faithfulness (precision) and F1 of the summary against both sources, measured on atomic facts rather than surface text overlap.
- **E2 — STT provider benchmark:** local whisper.cpp vs Google Cloud STT on word error rate (WER), domain-term error rate (TER), latency and cost.
- **E3 — question psychometrics:** classical test-theory item analysis (p-value difficulty, point-biserial discrimination) over recorded quiz attempts, with a rejection rule and a closed LLM regeneration loop.

Relevance

- **Who benefits.** Students in technical programs preparing for exams and retakes — anyone who needs to review 20+ hours of lecture video in days, not weeks. The same pipeline applies to corporate training recordings, conference talks, and MOOC back-catalogs.
- **Why the spoken channel matters.** In the course used as the dataset, the reconstructed spoken content of a single lecture contains hundreds of atomic facts that are absent from the slides (334 transcript-only claims out of 526 for lecture 6). Tools that summarize only the deck — or

only the audio — silently discard one of the two sources. This project treats "spoken but not on the slides" as the value to preserve and measure.

- **Why now.** Long-context LLMs with structured outputs make multi-source grounded summarization practical, and local STT (whisper.cpp) makes transcription free and private. What is still missing — and what this thesis contributes — is *evaluation*: n-gram metrics like ROUGE cannot tell whether a fact survived summarization, and LLM-based summaries need faithfulness guarantees before students can trust them. Claim-level evaluation with a calibrated LLM judge addresses exactly this.
- **Beyond summaries.** Passive reading is a weak learning signal. The quiz layer closes the loop — and its psychometric filter addresses a real weakness of LLM-generated quizzes: many generated questions are trivially easy or fail to discriminate between students who understood the lecture and those who did not.

Input and output

Input:

- **Lecture video** — MP4, H.264, 1920×1080 @ 60 fps, 1.5–3.5 hours per lecture (the dataset totals ≈ 21.3 hours over 9 lectures). Only the audio track is used: it is extracted with ffmpeg to 16 kHz mono PCM WAV — the shared input format for both STT providers, so recognition quality is comparable.
- **Slide deck** — text-based PDF (LaTeX/beamer), 21–87 pages per lecture (619 pages total across the course). Text is extracted per page with pypdf; scanned/image-only decks are out of scope (no OCR).

Intermediate representations (all stored per lecture, on disk):

- `transcription.json` / `.md` — unified transcript: segments and words with start/end timestamps and text (a provider-independent `Transcript` / `Segment` / `Word` schema; ~46,000 words of raw ASR for a 3.2-hour lecture).
- `transcription_reconstructed.md` — LLM-cleaned transcript: filler, Q&A, administrative talk and off-topic speech removed, recognition errors repaired, spoken math rendered as LaTeX (46k → 8.6k words for lecture 6, with *more* extractable atomic facts, not fewer — see Experiments).
- `slides_reconstructed.md` — LLM-cleaned slide text (beamer overlay duplicates collapsed, math repaired).

Output:

- `conspect.md` — the structured summary: Markdown with LaTeX math, every statement tagged with its source (`slide` / `transcript` / `both`); ~3,200 words for lecture 6.
- `quiz.json` — 5 single-choice questions; a pydantic schema enforces exactly one correct option per question; each option carries an explanation.
- `e1/e1.json` — the evaluation metrics for the lecture (plus cached `claims.json` and per-direction judgment files).
- A **web application** (Flask) serving course → lecture → conspect → quiz → results → item statistics; attempts are stored in SQLite keyed by a content-hash question ID.

Metrics

E1 — summary quality (claim level). All three texts (reconstructed slides, reconstructed transcript, conspect) are decomposed by an LLM into atomic self-contained claims, each labelled with salience (`core` / `supporting` / `aside`). A single entailment engine — top-k embedding retrieval followed by a strict batched LLM-judge NLI decision — is run in three directions:

metric	definition
Coverage (recall)	share of source claims (slides + transcript) entailed by the summary; also reported per source and restricted to `core` claims
Faithfulness (precision)	share of summary claims entailed by the sources (1 − hallucination/distortion rate)
F1	harmonic mean of coverage and faithfulness
Transcript-only coverage	of the spoken claims *not* entailed by the slide claims, the share the summary kept — the "summary > deck" thesis in one number

Since the judge is itself an LLM, its verdicts are validated against manual labels before the metrics are trusted (see Validation). Achieved so far on the development lecture: coverage 62.5%, faithfulness 86.5%, F1 72.6%, transcript-only coverage 48.5% — targets for the remaining lectures are F1 ≥ 70% and faithfulness ≥ 85%, which the development results show are attainable.

E2 — STT quality. WER (word error rate) against a manually transcribed golden segment; **TER** (term error rate) — the share of domain terms, harvested from the slides, missing from the transcript under fuzzy matching (measurable without any manual annotation); plus latency and cost per hour. Achieved: WER 2.3% (whisper.cpp large) vs 16.3% (Google STT) on the golden set.

E3 — question quality (classical test theory). Per question: **difficulty** p = share of correct responses; **discrimination** r_pb = point-biserial correlation between answering the item correctly and the *rest score* (total score excluding the item). Rejection rule (with ≥ 5 attempts): retire a question if $p \notin [0.30, 0.85]$ or $r_pb < 0.15$; retired questions are archived and LLM-regenerated conditioned on their measured stats and miss direction. Success criterion: after one retire-regenerate cycle, the share of questions inside the target difficulty band increases. With $n \approx 15$ respondents the estimates are noisy — the deliverable is the working methodology and the closed loop, with the sample-size caveat stated explicitly.

Data

The dataset is a real university course: **"Deep Learning and Computer Vision" (Harbour Space, 2026, lecturer Sergey Nikolenko)** — collected by the author from the public course page and lecture recordings.

- **9 lectures**, each an object consisting of: a video recording (MP4, 1920×1080@60, H.264; durations 1h 28m – 3h 30m; ≈ 21.3 hours total) and the matching slide deck (text-based beamer PDF, 21–87 pages; 619 pages total). Topics span classical ML introduction through CNNs, Transformers, GANs and diffusion models — dense mathematical content (formulas, theorems, architectures), which stresses both STT (terminology) and summarization (LaTeX, precise claims).
- **Golden STT set:** a 19.5-minute segment of lecture 6 (2:01:41–2:21:15) manually transcribed into a reference transcript (~2,600 words), with the matching whisper.cpp and Google STT outputs for the same segment.
- **Judge-calibration labels:** 40 (claim, premises) pairs sampled across all three judgment directions of lecture 6, hand-labelled as entailed / not entailed.

- **Quiz-attempt data:** responses from the course group (~15 students) collected through the web app into SQLite (attempts, per-question answers). This dataset is still being collected; its small n is the stated limitation of E3.
- **Special traits.** Lectures are long-form and single-speaker with occasional student questions (removed during reconstruction); the audio contains accented English and heavy domain terminology (biased against generic STT language models — visible in the Google STT WER); slide decks are beamer PDFs with overlay duplication (handled by slide reconstruction); one lecture (lecture 4) has an unusually short deck (21 pages), so per-lecture metric variance is expected.

Links to datasets

- Course page (videos + slides) — https://www.sergeynikolenko.ru/en/courses/harbourbcn26
- Collected dataset, golden set and calibration labels — in the project repository (`data/`): https://github.com/homomorfism/autosummary

Validation

There is no trained model in the pipeline (all learned components are frozen pretrained models), so validation targets the *measurements* rather than a train/test split:

- **LLM-judge calibration (E1).** The judge's verdicts are validated against 40 hand-labelled pairs stratified over the three judgment directions: 39/40 agreement (97.5%), Cohen's $\kappa = 0.95$, **zero false positives** and one false negative. Zero FP means faithfulness is not inflated by a lenient judge; the single FN means coverage-style numbers are slightly conservative. With $n = 40$ the 95% CI on agreement is roughly 87–100%, so this is reported as "$\kappa \approx 0.95$ on a 40-pair sample", and recalibration is repeated after any judge-prompt or judge-model change.
- **Development / held-out split by lecture (E1).** The pipeline (prompts, retriever, top-k, reconstruction stage) was iterated on lecture 6 only. The finalized configuration is then run unchanged on the remaining 8 lectures, which serve as a held-out set; results are reported per lecture and aggregated.
- **Golden reference (E2).** WER/TER are computed against the manually transcribed 19.5-minute golden segment, identical audio for both providers (same ffmpeg 16 kHz mono extraction), so the comparison isolates the recognizer.
- **Psychometrics guardrails (E3).** Item statistics are computed only from attempts with ≥ 5 responses per question; discrimination uses the rest score (the item's own contribution excluded) to avoid self-correlation; the $n \approx 15$ sample-size caveat is reported alongside every estimate.

Solutions overview

FActScore: Fine-grained Atomic Evaluation of Factual Precision (Min et al., 2023) — https://arxiv.org/abs/2305.14251
Introduced decomposing generated text into atomic facts and checking each against a knowledge source. E1 adopts this claim-level paradigm but runs it in *both* directions (coverage and faithfulness are the same entailment engine with the source of truth and the text under test swapped) and adds salience labels so that deliberately dropped asides do not count as coverage failures.

QAGS: Asking and Answering Questions to Evaluate Factual Consistency (Wang et al., 2020) — https://arxiv.org/abs/2004.04228
An early demonstration that ROUGE/BLEU do not measure factual consistency of summaries, using QA as the probe. Motivates this project's move away from n-gram overlap; E1 replaces the QA probe with direct NLI-style entailment over retrieved claims.

SummaC: NLI Models for Summary Inconsistency Detection (Laban et al., 2022) — https://arxiv.org/abs/2111.09525
Showed how to apply sentence-level NLI to document-level faithfulness. E1's `entails(claim, premise_set)` follows the same NLI framing but replaces a fixed NLI model with top-k embedding retrieval + an LLM judge, which handles LaTeX-heavy, multi-premise lecture claims.

G-Eval: NLG Evaluation Using GPT-4 (Liu et al., 2023) — https://arxiv.org/abs/2303.16634
Representative of LLM-as-judge evaluation and its pitfalls. This project takes the pitfalls seriously: the judge is calibrated against human labels (κ = 0.95, 0 false positives) before its numbers are used.

Robust Speech Recognition via Large-Scale Weak Supervision — Whisper (Radford et al., 2022) — https://arxiv.org/abs/2212.04356, local runtime **whisper.cpp** — https://github.com/ggml-org/whisper.cpp
The local STT backbone (E2 baseline candidate). The project wraps it behind an `STTProvider` abstraction with anti-hallucination flags tuned for multi-hour lectures, and benchmarks it against Google Cloud Speech-to-Text (long-running recognition).

RAGAS: Automated Evaluation of Retrieval Augmented Generation (Es et al., 2023) — https://arxiv.org/abs/2309.15217
Popularized reference-free precision/recall-style LLM evaluation (faithfulness / answer relevance) for generated text over retrieved context — the closest engineering analogue to E1's retrieve-then-judge design.

Commercial lecture-note tools (Otter.ai — https://otter.ai, NoteGPT — https://notegpt.io)
Transcribe and summarize audio or video, but (a) use a single source rather than slides+speech jointly, (b) provide no grounding tags or faithfulness measurement, and (c) have no feedback loop on question quality. These are the practical baselines the thesis differentiates against.

Experiments

Baseline

The baseline is the first working end-to-end pipeline, deliberately low-effort in each component:

- **Summary:** conspect generated directly from the **raw ASR transcript** (46k words, with filler, Q&A and off-topic speech) + extracted slide text.
- **Evaluation:** claim decomposition without salience labels; retrieval with `all-MiniLM-L6-v2`, top-k = 8; `deepseek-v4-pro` judge.
- **STT reference point:** Google Cloud STT (cloud long-running recognition) vs whisper.cpp on the golden set.
- **Conceptual reference point:** a slides-only summary, which by construction scores 0% on transcript-only coverage.

Baseline metrics and results

Lecture 6 (development lecture), whisper.cpp transcript:

metric	baseline
coverage	53.9%
coverage: slides / transcript	70.9% / 48.8%
faithfulness	69.4%
F1	60.7%
transcript-only preserved	37.2% (of 349 claims)

STT golden set (19.5 min, lecture 6): **whisper.cpp WER 2.3%** vs **Google STT WER 16.3%** (TER shows the same ordering) — whisper.cpp is selected as the pipeline default; it is also free and local.

Main model

The main pipeline keeps the same skeleton and upgrades the stages that the baseline showed to be the bottleneck (ASR noise contaminating both the summary and the measurement):

1. **Transcript reconstruction** (`app/reconstruct_transcript.py`) — a new LLM stage that turns raw ASR into a clean lecture text: removes filler/Q&A/administrative/off-topic speech, repairs recognition errors, renders spoken math as LaTeX. 46k → 8.6k words on lecture 6, while *increasing* the number of extractable atomic claims (526 vs 449) and not shrinking the transcript-only pool (334 vs 294) — evidence the compression is cleaning, not hidden summarization.
2. **Conspect regenerated** from the reconstructed transcript + reconstructed slides (structured-output LLM call; source tagging preserved by the pydantic schema).
3. **Evaluation upgrades** — salience labels (`core` / `supporting` / `aside`) at decomposition; retriever `BAAI/bge-small-en-v1.5`; top-k = 16 so compound claims gather all their supporting premises. All stages cache to `<lecture>/e1/` keyed on a settings fingerprint.
4. **Quiz layer (E3)** — quiz generation from the conspect, web app collection of attempts, item analysis (p, r_pb vs rest score), rejection rule, and LLM regeneration conditioned on measured stats.

Main model metrics and results

Lecture 6, same judge and transcript provider as the baseline (middle column = intermediate run isolating the evaluation-side changes):

metric	baseline	+ clean decomposition	**main (reconstructed)**
coverage	53.9%	51.5%	**62.5%**
coverage (core claims only)	—	64.9%	**68.6%**
coverage: slides / transcript	70.9% / 48.8%	63.3% / 49.0%	70.1% / **60.3%**
faithfulness	69.4%	73.6%	**86.5%**
F1	60.7%	60.6%	**72.6%**
transcript-only preserved	37.2% (of 349)	37.4% (of 294)	**48.5%** (of 334)
transcript-only preserved (core)	—	49.2% (of 120)	**52.4%** (of 168)

Judge validity behind these numbers: 39/40 agreement with manual labels, κ = 0.95, 0 false positives (so faithfulness is not judge-inflated).

The three changes between the middle and right columns (better conspect, fairer measurement, larger top-k) are currently confounded; an ablation separating them is planned. Remaining work: run the frozen pipeline over the other 8 lectures (held-out E1 numbers), complete the E2 latency/cost table, and collect the student attempts needed to run the E3 retire-regenerate cycle on live data.

Code repository

https://github.com/homomorfism/autosummary — Python (uv), `app/` package with the pipeline stages as CLI modules, `docs/experiments/` with experiment write-ups, `data/` with the collected course, golden set and cached evaluation artifacts.

Usage

The end product is a **course web app** for students:

```
uv run python -m app.web        # http://127.0.0.1:8000
```

Any `<course>/<lecture>/` directory containing a `conspect.md` appears automatically: the student opens the lecture, reads the conspect (server-rendered Markdown with MathJax for LaTeX), takes the 5-question quiz, and sees their result; every attempt feeds the E3 item statistics visible on the per-lecture `/stats` page, which in turn drive question retirement and regeneration — so the quiz improves as more students use it.

Content producers (course staff) run the pipeline per lecture:

```
uv run python -m app.transcribe <lecture-dir>              # STT (whisper.cpp / Google)
uv run python -m app.reconstruct_transcript <lecture-dir>  # raw ASR → clean transcript
uv run python -m app.conspect <lecture-dir>                # transcript + slides → conspect.md
uv run python -m app.quiz <lecture-dir>                    # conspect.md → quiz.json
uv run python -m app.quality <lecture-dir> --regenerate    # psychometrics + closed loop
```

All working results are stored in transparent formats: Markdown (transcripts, conspect), JSON (transcript with timestamps, quiz, E1 metrics and claims), SQLite (quiz attempts and retired-question archive) — so every artifact is inspectable and versionable. When deployed as a service, the same pipeline runs behind an upload endpoint (video + PDF in, lecture page URL out); the longer-term product direction is students uploading and sharing course materials across universities.